Anteater Poker Software Spec

Version 0.3

Authors: Prithvi Binu, Justin Wu, Katrina Gong, William Culley, Stefano Pinna Segovia, Rishab Senthil Kumar, Duc Tran

Affiliation: University of California, Irvine
Team Name: 99%

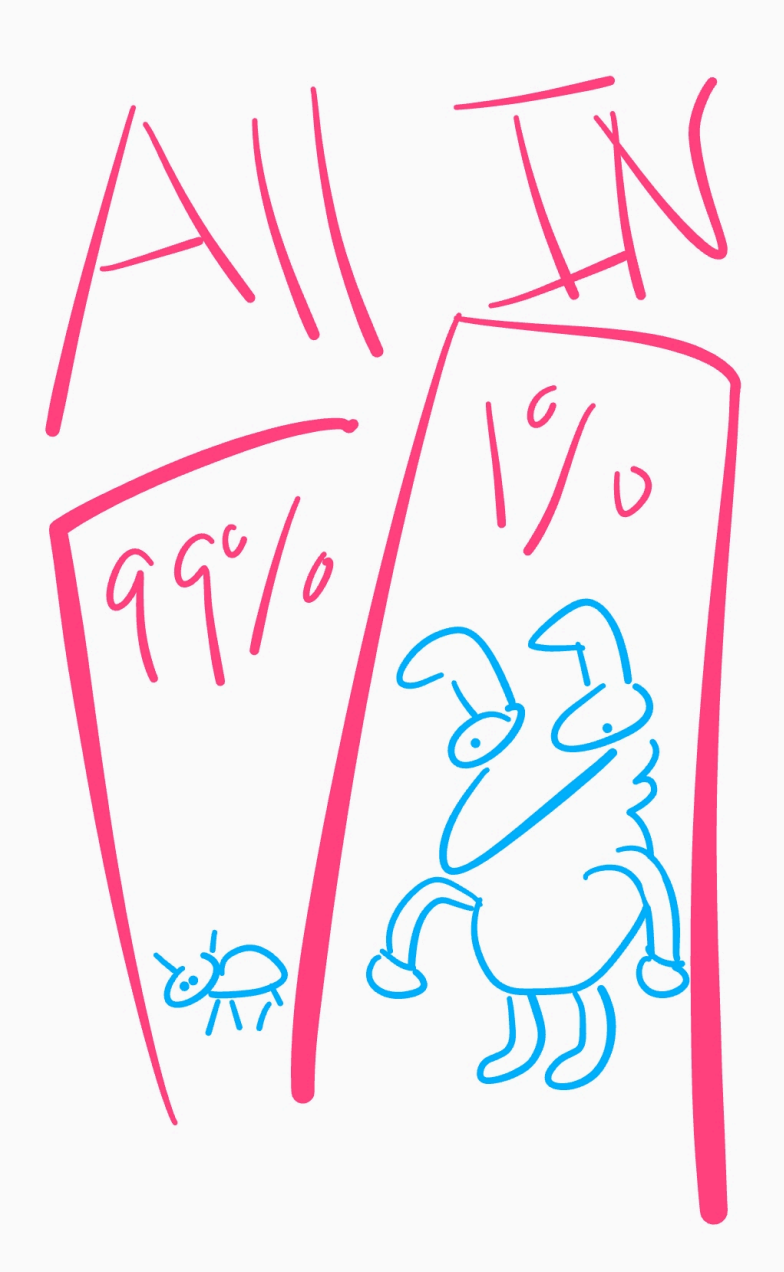


Table of Contents

Glossary

Term	Definition
All in	When a player bets their remaining chips.
Big Blind	A mandatory bet placed by the player 2 positions to the left of the dealer before any cards are dealt.
Call	Match the current highest bet to stay in the round.
Check	If no bet has been made, pass the turn to the next active player without betting.
Community	The 5 cards that are shared by every player. These cards are revealed in stages.
Dealer	The person that typically deals the cards and shuffles in a typical poker game.
Flop	The stage where there is a betting round after the first 3 community cards are revealed.
Fold	Throw in cards and give up your hand for the current game, forfeiting any chips that are already bet.
Hole	Two private cards that only the player can see, which will be revealed after betting.
Pot	The total amount of chips bet by all players in the current hand.
Raise	Increase the current bet by a minimum raise amount.
River	The stage where there is a betting round after the 5th community card is revealed.
Preflop	The stage where there is an initial betting round where no community cards are revealed.
Showdown	Where every card is revealed and the winner is chosen out of the remaining players.
Small Blind	A mandatory bet placed by the player 1 position to the left of the dealer before any cards are dealt.
Turn	The stage where there is a betting round after the 4th community card is revealed.

1/2. Poker Client/Server Software Architecture Overview

1/2.1 Main data type and structures

Main data type: uds.h (unified data structure)
This data structure will be shared between the client and the server. Ensuring that both client and server are using the same data structure.

Type	Values	Purpose
MoveType	FOLD, CHECK, CALL, RAISE, ALL_IN	Player betting actions
Stage	PREFLOP, FLOP, TURN, RIVER	Current betting round
Suit	UNKNOW_S, HEARTS, DIAMONDS, CLUBS, SPADES	Card suit
Rank	UNKNOW_R, TWO … ACE	Card face value
PlayerStatus	EMPTY, CONNECTED, READY, PLAYING, SPECTATING, FOLDED, ALL_IN, DISCONNECTED	Seat lifecycle state
Rankings	HIGH_CARD … ROYAL_FLUSH	Poker hand classification

Core Structs

Card — a single playing card.

```
typedef struct {
  uint8_t suit;   // Suit enum
  uint8_t rank;   // Rank enum
  uint8_t value;  // Numeric value for comparison
} Card;
```

Deck — the 52-card deck plus 4 reserved special-card slots.

```
typedef struct {
  Card    cards[56];  // Standard deck + 4 special cards
  uint8_t top;        // Index of the next card to deal
} Deck;
```

PlayerHand — a player's two private hole cards.

```
typedef struct {
  Card hand[2];
} PlayerHand;
```

Player — complete player state at the table.

```
typedef struct {
  uint8_t      id;
  char         name[32];
  PlayerHand   hand;
  uint32_t     chips;
  uint32_t     current_bet;
  PlayerStatus status;
  bool         has_cards;
} Player;
```

GameState — the full, authoritative game snapshot broadcast to every client.

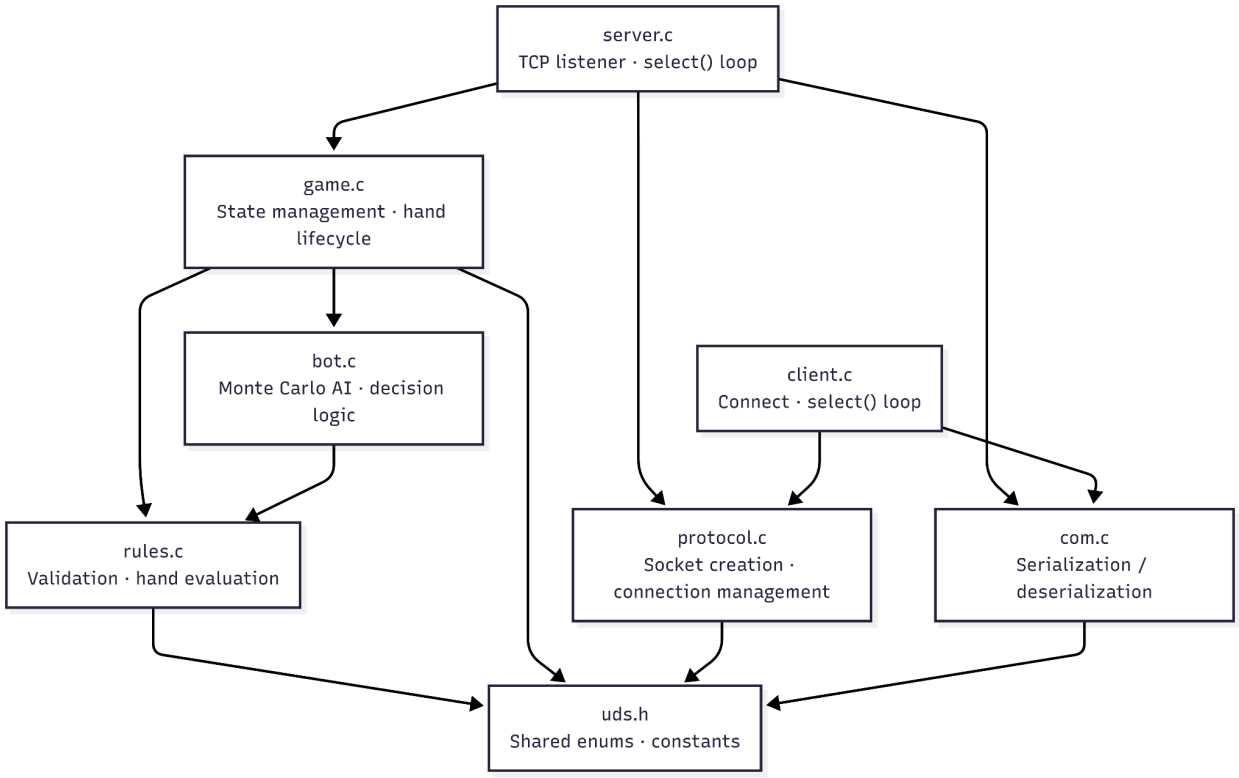
```
typedef struct {
  Player   players[6];
  uint8_t  playerCount;
  Card     community[5];
  uint8_t  communityCount;
  Stage    stage;
  uint8_t  currentPlayer;
  uint8_t  dealerIndex;
  uint32_t pot;
  uint32_t currentBet;
  uint32_t minRaise;
  bool     handPlaying;
  bool     acted[6];    // Whether each player has acted this round
} GameState;
```

1/2.2 Major software components

Module	Files	Responsibility
Server	server.c	TCP listener, select() event loop, client lifecycle
Client	client.c	Connects to server, forwards user input, renders game state
Game Engine	game.c / game.h	Deck/player init, card dealing, move application, hand lifecycle

Rules Engine	rules.c / rules.h	Move validation, hand evaluation, winner determination
Bot AI	bot.c / bot.h	Monte Carlo simulation, decision logic, bot turn execution
Protocol	protocol.c / protocol.h	Socket creation, connection management, send/receive helpers
Codec	com.c / com.h	Serialization and deserialization of all wire messages
Data Types	uds.h	Shared enumerations and constants (no implementation)

Hierarchy chart:



1/2.3 Module interfaces

1. API of major module functions

game.h — Game Engine

```
void initDeck(Deck *deck)                    // Populates the deck with 52 standard playing cards

void initPlayer(Player *p, uint8_t id,       // Zero-initializes a player seat
        const char *name, uint32_t chips)    // Assigns id, display name, and starting chip count
```

```
void shuffle(Deck *deck)                        // Fisher-Yates shuffle of the entire deck

void shuffleRemaining(Deck *deck, int count)    // Shuffles only the undealt portion of the deck
                                                // Used by the bot to randomize unknown cards

Card deal(Deck *deck)                           // Returns the next card from the top of the deck
                                                // Advances the deck's top index by 1

void dealHoles(GameState *gs, Deck *deck)       // Deals 2 hole cards to every
PLAYER_READY seat
                                                // Sets those players to PLAYER_PLAYING

void apply(GameState *gs, uint8_t playerID,     // Applies a validated move to the game state
        MoveType move, uint32_t amount)         // Updates chips, current_bet, pot, and player
status

void award(GameState *gs)                       // Distributes the pot to the winner(s)
                                                // Splits evenly among tied winners

void resetHand(GameState *gs)                   // Clears bets, cards, and acted[] flags
                                                // Advances the dealer button to the next active seat

void newHand(GameState *gs, Deck *deck)         // Shuffles deck, deals holes, posts blinds
                                                // Sets stage to PREFLOP and handPlaying to true

void advance(GameState *gs, Deck *deck)         // Deals the next community cards for the
current stage
                                                // Calls award() automatically after the RIVER
```

```
void processMove(GameState *gs, Deck *deck, uint8_t playerID)  // Post-apply bookkeeping after each move
                                                               // Checks round end, advances stage, rotates currentPlayer

bool tryMove(GameState *gs, Deck *deck,                        // Validates then applies a move
        uint8_t playerID, MoveType move, uint32_t amount) // Returns false if the move is illegal; state unchanged

bool resolveNoAct(GameState *gs, Deck *deck)                   // Fast-forwards to showdown when no player can act
                                                               // Returns true if the game was resolved early
```

---

rules.h — Rules Engine

```
bool validate(const GameState *gs, uint8_t playerID,          // Returns true if the move is legal given current state
        MoveType move, uint32_t amount)                    // Checks bet amounts, player status, and turn order

int evaluateHand(const GameState *gs, const PlayerHand *hand)  // Finds the best 5-card score from 2 hole + up to 5 community cards
                                                               // Returns a packed int for direct comparison between players

int score5(Card cards[5])                                      // Classifies and scores exactly 5 cards
                                                               // Bits 23–20 = hand type (0–9), bits 19–0 = tie-breaker ranks

int findActive(const GameState *gs,                            // Fills activeIDs[] with seats still in the hand
        int activeIDs[], bool inclAllIn)                   // Returns the count of active players found

int nextActive(const GameState *gs,                            // Circular search for the next eligible seat after curr
        int curr, bool inclReady)                          // inclReady = true also considers PLAYER_READY seats
```

```
bool allPlayersWent(const GameState *gs)                    // Returns true when every active player has acted this round
                                                            // Used by processMove() to decide whether to advance the stage

void initBlinds(GameState *gs)                              // Deducts and posts the small and big blinds
                                                            // Sets currentPlayer to the seat after the big blind

int determineWinners(const GameState *gs, int winnerIDs[])  // Evaluates all active hands at showdown
                                                            // Fills winnerIDs[] and returns the number of winners
```

---

bot.h — Bot AI

```
bool isBot(const char *name)                                // Returns true if name matches a reserved bot name
                                                            // Reserved names: Alvin, Randy, Betty, Colleen, Minnie, Dumbo

void addBot(GameState *gs)                                  // Fills each empty seat with a bot player
                                                            // Bot names are assigned in random order

int reconstructDeck(const GameState *gs,                    // Builds a deck containing only cards not visible to the bot
                    const PlayerHand *hand, Deck *deck)     // Excludes community cards and the bot's own hole cards

double monteCarloSim(const GameState *gs, const Deck *deck,  // Runs simCount simulated hands from the current state
                     const PlayerHand *botHand, int simCount)  // Returns estimated win probability between 0.0 and 1.0

MoveType decide(const GameState *gs, const Deck *deck)      // Selects FOLD / CHECK / CALL / RAISE / ALL_IN
                                                            // Based on win probability thresholds: 50%, 65%, 75%
```

```
void botMove(const GameState *gs, const Deck *deck,          // Produces a complete action for the current bot seat
             uint8_t *botID, MoveType *move, uint32_t *amount) // Sets raise amount to a random value in [minRaise, maxRaise]

void sendMove(int socket, MoveType move, int amount)         // Encodes and transmits a bot action over the socket

void doBotTurn(GameState *gs, Deck *deck, ...)               // Executes bot turns in a loop until a human
                                                             // Calls botMove() then tryMove() for each consecutive bot seat
```

---

protocol.h — Network

```
void init_server(ServerState *state)                         // Zero-initializes all server state fields
                                                             // Sets running = true and clears all client slots

int create_socket(Client *client)                            // Opens a listening socket on port 10160
                                                             // Returns the file descriptor

void add_connection(ServerState *state, Client *client)      // Accepts an incoming connection and assigns it a seat
                                                             // Sets client status to PLAYER_CONNECTED

void handle_client_communication(ServerState *state,Client *client)      // Reads one message from a client socket

void remove_client(ServerState *state, Client *client)       // Disconnects a client but still storing their info PLAYER_DISCONNECTED

void cleanup_server(ServerState *state)                      // Closes all open sockets and frees server
```

```
void init_client_state(ClientState *client)                // Zero-initializes all client state fields

int connect_to_server(const char *hostname, int port)      // Establishes a connection to the server
                                                           // Returns the socket file descriptor

void send_data_to_server(ClientState *client,const char *data)  // Encodes and transmits a PlayerAction
to the

void receive_data_from_server(ClientState *client)         // Reads and decodes an incoming
GameState from the server
                                                   // Updates the local game snapshot

void handle_user_input(ClientState *client)                // Reads a command from stdin and sends it as a
PlayerAction
```

---

com.h — Encode/Decode

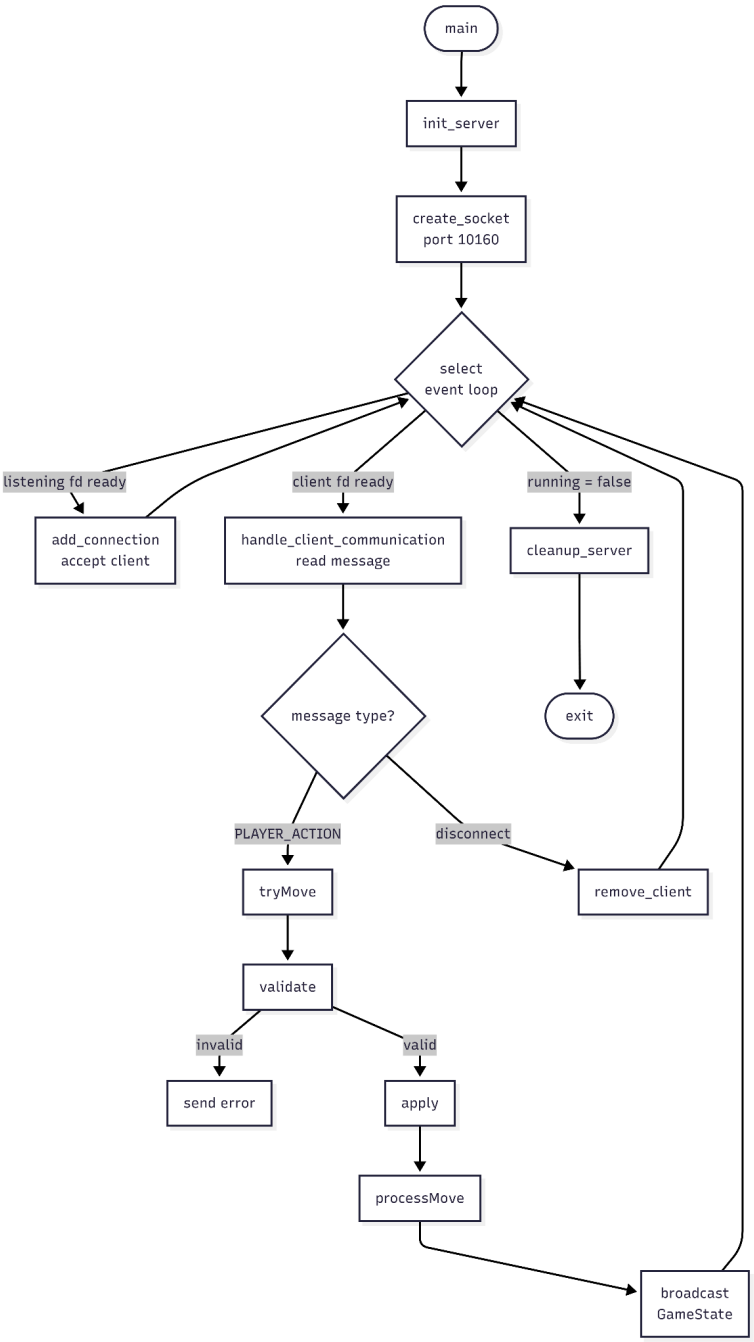
```
bytes encode_client_data(PlayerAction *action)             // encode a PlayerAction into wire bytes
                                                   // Layout: playerID (u8) · move (u8) · amount (u32 big-endian)

PlayerAction decode_client_data(bytes buf)                 // decode wire bytes back into a PlayerAction

bytes encode_server_data(GameState *gs)                    // encode the full GameState into a wire

GameState decode_server_data(bytes buf)                    // decode a wire message back into a
GameState
```

1/2.4 Overall program control flow



1.5 Automated Client: Poker Bot

Overview

The bot subsystem allows AI-controlled players to occupy seats at the table. Up to six bot names are reserved — Alvin, Randy, Betty, Colleen, Minnie, Dumbo — and addBot() fills any empty seat from

that pool in random order. isBot(name) is checked server-side before each turn to decide whether to call doBotTurn() instead of waiting for a human client message.

Monte Carlo Win Estimation (monteCarloSim)

The bot does not use pre-computed hand tables. Instead, it estimates its win probability by simulating 1 000 complete hands from the current state:

```
for each simulation (1 … 1000):
  1. reconstructDeck() — collect every card not visible to the bot
                         (excludes community cards and bot's own hole cards)
  2. shuffleRemaining() — randomize the unknown cards
  3. Deal remaining community cards from the unknown deck
  4. Deal 2 hole cards to every other active player from the unknown deck
  5. evaluateHand() for each active player
  6. If bot is among the winners: win_count += 1 / winner_count
                                  (fractional credit for split pots)
return win_count / 1000   →  estimated win probability  (0.0 – 1.0)
```

Decision Logic (decide)

```
P = monteCarloSim(1000)
if bot owes chips to call:
  if chips < amount owed:
    P > 0.50  →  ALL_IN
    else      →  FOLD
  else:
    P > 0.75  →  RAISE
    P > 0.50  →  CALL
    else      →  FOLD
if no chips owed (can check free):
  P > 0.65  →  RAISE
  else      →  CHECK
```

Raise Sizing (botMove)

When decide() returns RAISE, the raise amount is chosen randomly in the range [minRaise, maxRaise], where maxRaise is the bot's remaining chips after covering its share of the current bet. If the bot cannot meet minRaise, it falls back to CALL or CHECK.

Bot Turn Execution (doBotTurn)

The server calls doBotTurn() immediately after updating currentPlayer. The function loops — calling botMove() then tryMove() — for as long as the new currentPlayer is also a bot, so multiple consecutive bot seats are resolved before the next network broadcast.

Future: Special Card Bots (specialCards.h)

Stubs exist for special-variant actions (swapCards, redrawCards, revealComCard, revealOppCard, swapOppCards) that will require extending the bot decision model to reason about information-revealing and card-manipulation moves.

3. Installation

3.1 System requirements, compatibility

1. Operating System: RHEL8 64-bit Linux, C11 (-std=c11), <sys/socket.h> <netinet/in.h>,<sys/select.h>,<netdb.h>.
2. Network: can connect to one of the hosts below with port:10160
    a. crystalcove.eecs.uci.edu: IP address 128.200.85.14
    b. • zuma.eecs.uci.edu: IP address 128.200.85.17
    c. • bondi.eecs.uci.edu: IP address 128.200.85.22
    d. • laguna.eecs.uci.edu: IP address 128.200.85.23
    e. • balboa.eecs.uci.edu: IP address 128.200.85.25 (VPN)
    f. • doheny.eecs.uci.edu: IP address 128.200.85.26 (VPN)

3.2 Unpacking and Configuration

1. Download the user package from the server (team16@bondi.eecs.uci.edu/repos/poker.git)
2. Extract the user package using the following commands:
    % gtar xvzf BinaryArchive.tar.gz
    % evince poker/doc/Poker_UserManual.pdf
    % poker/bin/server &
    % poker/bin/poker

- The above commands allow the user to extract the poker folder using [gtar] and view the documentation using the document viewer with [evince]. It will also execute the online poker program.

3.3 Building, compilation, installation

1. Execute the following commands:
    %make
    %cd bin

—For running the server:
%cd server
%./server
-For running client
%cd client
%./client bondi 10160
Remove compiled source file and binary
%make clean

- The above commands allow the user to remove the extracted poker file, and the user package.

4. Documentation of packages, modules, interfaces

4.1 Detailed description of data structures

- Critical snippets of source code

```
//struct that current game state (broadcast to all players)
// excludes the deck and player cards which are server-side
typedef struct {
    Player players[MAX_PLAYERS];
    uint8_t playerCount;
    //uint8_t turn;
    Card community[5];
    uint8_t communityCount;  // how many community cards are revealed

    uint8_t stage;           // use Stage enum
    uint8_t currentPlayer;
    uint8_t dealerIndex;
    //uint8_t yourIndex;

    uint32_t pot;
    uint32_t currentBet;     // bet needed to match
    uint32_t minRaise;

    bool handPlaying; //for starting new hands after reset
    bool acted[MAX_PLAYERS];
} GameState;
```

- The GameState stores all major game data on the server. This data is meant to be broadcast to all clients. Only valid actions can mutate the GameState (verified through the validate() function).

```
//struct of player, contains 8 bits id, 32 bytes name, 32 bits chips, 32
bits current bet, 8 bits status and 8 bits has_cards.
typedef struct {
    uint8_t id;
    char name[MAX_NAME_LENTH];
    PlayerHand hand;
    uint32_t chips;
    uint32_t current_bet;

    uint8_t status;
    uint8_t has_cards;

}   Player;
```

- The Player struct contains a player's private cards (in hand) as well as their status and number of chips owned. An array of players is contained in the GameState.

```
//struct of deck, contains 52 cards and N special cards, 8 bits top to
indicate the top card index in the deck.
typedef struct{
    Card cards[DECK_SIZE + SPECIAL_CARDS];
    uint8_t top;
} Deck;
```

- The Deck is stored within the server and not exposed by GameState. Separating the Deck might help against cheaters. The deck contains a top card pointer that can be used to reset it (deck->top = 0).

```
typedef struct {
    uint8_t suit;
    uint8_t rank;
    uint8_t value;
} Card;
```

- The Card struct defines a poker card by its suit, rank, and value (may be used by evaluateHand).

4.2 Detailed description of functions and parameters

- Function prototypes and brief explanation

Deck Functions

- void initDeck(Deck *deck);
  - Initializes the deck with all standard cards and resets the top index.
  - Parameters:
    - deck: deck to initialize
  - Returns: Nothing

- void shuffle(Deck* deck);
  - Randomly shuffles the deck.
  - Resets the deck top index back to the first card.
  - Parameters:
    - deck: deck to shuffle
  - Returns: Nothing

- Card deal(Deck* deck);
  - Deals one card from the top of the deck.
  - Advances the deck top index after dealing.
  - Parameters:
    - deck: deck to deal from
  - Returns: The dealt card

- void dealHoles(GameState* gs, Deck* deck);
  - Deals two private hole cards to each ready player.
  - Changes ready players into playing players.
  - Parameters:
    - gs: current game state
    - deck: deck to deal from
  - Returns: Nothing

Hand Scoring

- int evaluateHand(const GameState* gs, const PlayerHand* hand);
  - Hand evaluation function for server(showdown) and bot.
  - Combines the community cards and player's private cards to determine strength.
  - Parameters:
    - gs: current game state
    - hand: player's private hand (hole cards)
  - Returns: Integer score representing hand strength

Move Handling

- bool validate(const GameState* gs, uint8_t playerID, MoveType move, uint32_t amount);
  - Checks if a move is valid
  - Verifies turn order and determines if the move can be made.
  - Parameters:
    - gs: current game state
    - playerID: player attempting the move
    - move: requested move type
    - amount: amount used for raises or all in moves
  - Returns: true for valid move

- void apply(GameState* gs, uint8_t playerID, MoveType move, uint32_t amount);
  - Applies a validated move to the game state.
  - Updates player chips, current bets, pot size, player status, and flags.
  - Parameters:
    - gs: current game state
    - playerID: player making the move
    - move: move to apply
    - amount: amount used for raises or all in moves
  - Returns: Nothing

- bool tryMove(GameState* gs, Deck* deck, uint8_t playerID, MoveType move, uint32_t amount);
  - Move handler used by the server.
  - Validates the move, applies it, and processes the resulting game flow.
  - Parameters:
    - gs: current game state
    - deck: current deck
    - playerID: player attempting the move
    - move: requested move type
    - amount: amount used for raises or all in moves
  - Returns: true if the move was accepted, false otherwise

Helper Functions

- int findActive(const GameState *gs, int activeIDs[], bool inclAllIn);
  - Find players who are still active in the hand.
  - Can optionally include all in players, who cannot act but can still win the pot.
  - Parameters:
    - gs: current game state
    - activeIDs: output array for active player indexes
    - inclAllIn: whether to include PLAYER_ALL_IN players
  - Returns: Number of matching players found

- int nextActive(GameState* gs, int curr, bool inclReady);
  - Finds the next matching player after the current player.
  - Can optionally include ready players for setup and reset logic.
  - Parameters:
    - gs: current game state
    - curr: current player index
    - inclReady: whether to include PLAYER_READY players
  - Returns: Next matching player index, or -1 if none exists

Game Flow

- void initBlinds(GameState* gs);
  - Posts the small blind and big blind.
  - Sets the current bet, minimum raise, and first player to act.
  - Parameters:
    - gs: current game state
  - Returns: Nothing

- void newHand(GameState* gs, Deck* deck);
  - Starts a new poker hand.
  - Shuffles the deck, deals hole cards, sets the stage to preflop, and posts blinds.
  - Parameters:
    - gs: current game state
    - deck: deck used for the hand
  - Returns: Nothing

- void advance(GameState* gs, Deck* deck);
  - Advances the hand to the next stage.
  - Deals the flop, turn, or river, or awards the pot after the river.
  - Parameters:
    - gs: current game state
    - deck: deck to deal community cards from
  - Returns: Nothing

- void processMove(GameState* gs, Deck* deck, uint8_t playerID);
  - Resolves the game state after a move has been applied.
  - Checks for one player left, all in runout, advancing stage, or next turn.
  - Parameters:
    - gs: current game state
    - deck: deck used for possible phase advancement
    - playerID: player who just moved
  - Returns: Nothing

- void award(GameState* gs);
  - Awards the pot at showdown.
  - Compares active players' hand scores and splits the pot in case of a tie.
  - Parameters:
    - gs: current game state
  - Returns: Nothing

- void resetHand(GameState* gs);

- Resets gamestate values after hand ends (showdown).
- Clears player cards, current bets, community cards, pot, current bet, and acted flags.
- Parameters:
  - gs: current game state
- Returns: Nothing

Server main

`int main(int argc, char *argv[])`

- Entry point for the server process

- argv : unused; server always binds to PORT 10160

- Behavior:

- 1. Calls init_server() to zero-initialize ServerState

- 2. Calls create_socket() to open the TCP listening socket

- 3. Enters the select() event loop:

- - Monitors the listening socket and all connected client sockets

- - New connection on listen_fd → add_connection()

- - Data on client_fd → handle_client_communication()

- 4. When state.running == 0, exits the loop and calls cleanup_server()

- Returns 0 on clean exit

Client main

`int main(int argc, char *argv[])`

- Entry point for the client process

- argc : must be >= 3; prints usage and exits if fewer arguments are given

- argv : argv[1] = server hostname (string)

- argv[2] = server port number (string, converted with atoi)

- Behavior:

- 1. Calls init_client_state() to zero-initialize ClientState

- 2. Calls connect_to_server(hostname, port) to establish TCP connection

- 3. Enters the select() event loop monitoring communication between with server

- 4. When client.running == 0, exits loop and closes socket

- Returns 0 on clean exit

Server-Side Functions

`void init_server(ServerState *state)`

- Initializes all fields of the ServerState to safe defaults

- state : pointer to the ServerState to initialize

```
- For each of the 6 client slots:
-   - client_fd and sock_fd set to -1 (invalid / unused)
-   - connected set to 0
-   - id assigned as slot index
-   - address and name zeroed out
- Calls initDeck() to prepare state->deck for a fresh game
- Zeroes out state->game (full GameState memset)
int create_socket(Client *client)
- Creates, binds, and begins listening on a TCP socket at PORT 10160
- client       : unused parameter (reserved for future use)
- Creates a stream socket with os file descriptor
- Binds file descriptor to socket
- Calls listen() with a backlog of 5 pending connections
- Calls error() and exits on any failure
- Returns the listening socket file descriptor on success
void add_connection(ServerState *state, Client *client)
- Accepts an incoming matching protocol connection and assigns it to the first free client slot
- state        : pointer to ServerState; the accepted client is stored here
- client       : optional output pointer; if not NULL, filled with the new client's data
- Calls accept() on sucess connection.
-- On finding a free slot:
-   - Stores client_fd, sock_fd, connected = 1, address, and id (slot index + 1)
-   - Copies data into *client if client != NULL
-   - Prints "New client connected: <id>"
- If all 6 slots are full:
-   - Prints an error to stderr and closes the new fd (connection refused)
void handle_client_communication(ServerState *state, Client *client)
-data send a receive function, reads one message from a connected client, and sends a basic acknowledgement for
now
- state        : pointer to ServerState (used to remove the client on error)
- client       : pointer to the Client struct for the sender
- Reads up to 255 bytes of buffer size.
- On read error (n < 0): calls remove_client() and returns
- On EOF       (n == 0): calls remove_client() and returns
- On success:
-   - Prints "Received message from client <id>: <text>"
-   - Writes the fixed string "Message received" back to the client
-   - On write error: calls remove_client()
void remove_client(ServerState *state, Client *client)
- Disconnects a client and marks its slot as available
- state        : unused (reserved for future cleanup logic)
```

- client : pointer to the Client to disconnect

- disconnect clien once connection cut off

- Prints "Client disconnected: <id>"

-depend on the disconnect type. It either erase or keep client's fd

void cleanup_server(ServerState *state)

- Shuts down the server by closing all open connections and the listening socket

- state : pointer to the ServerState to clean up

- Calls remove_client() for every slot where connected == 1

- Closes state->listen_fd if it is a valid fd (>= 0), then sets it to -1

Client-Side Functions

void init_client_state(ClientState *client)

- Initializes all fields of ClientState to safe defaults

- client : pointer to the ClientState to initialize

- Sets socket_fd to -1 (no connection yet)

- Sets running to 1 (client starts in running state)

- Sets connected to 0

- Sets my_player_id to 0

- Zeroes out game (GameState) and input_buffer

int connect_to_server(const char *hostname, int port)

- Establishes a TCP connection to the server

- hostname : server host name or IP address string (e.g. "localhost")

- port : TCP port number to connect to (e.g. 10160)

- Creates a TCP socket with socket(AF_INET, SOCK_STREAM, 0)

- Resolves hostname to an IP address with gethostbyname()

-transform Port number into high-order byte

- Calls connect() to complete the connection

- Returns the connected socket fd on success

- Returns -1 on any failure (socket error, host not found, connect error)

- and prints the relevant error message to stderr

void send_data_to_server(ClientState *client, const char *data)

- Transmits a string of data to the server over the established TCP connection

- client : pointer to the ClientState; uses client->socket_fd

- data : null-terminated string to send (e.g. raw user input or encoded action)

- Calls write() with strlen(data) bytes

- On write error: prints error to stderr and sets client->running = 0

void receive_data_from_server(ClientState *client)

- Reads one message from the server and prints it to stdout

- client : pointer to the ClientState; uses client->socket_fd
- Reads up to 255 bytes into a local buffer
- On read error (n < 0): prints error to stderr, sets client->running = 0
- On EOF (n == 0): prints "Server disconnected", sets client->running = 0
- On success: prints "Received message from server: <text>"
- Note: GameState decode and UI update logic is not yet implemented here

void handle_user_input(ClientState *client)

- Reads a command from stdin, parses it, and sends it to the server
- client : pointer to the ClientState
- Declared in protocol.h but not yet called in client.c main loop
- (inline stdin handling is used directly in main instead)

void error(const char *msg)

- Prints a system error message and terminates the process
- msg : descriptive label prepended to the system error text
- Calls perror(msg) to print msg + the current errno description to stderr
- Calls exit(1) — the process does not return from this function
- more to implement

void decode(const char *data, GameState *game)

- Deserializes a raw byte string into a GameState
- data : pointer to incoming wire bytes
- game : pointer to the GameState to populate
- This function will decode the data from one payload

char *encode(const GameState *game, size_t buffer_size)

- Serializes a GameState into a newly allocated byte buffer
- game : pointer to the GameState to serialize
- buffer_size : maximum number of bytes to write
- This function will encode the data into one payload

Encoder and Decoder

void write_u8(uint8_t *buffer, uint32_t *offset, uint8_t val)

- Writes a single byte into the buffer at the current offset, then advances offset by 1
- buffer : destination byte array
- offset : pointer to the current write position; incremented by 1 after write
- val : the 1-byte value to write

void write_u32(uint8_t *buffer, uint32_t *offset, uint32_t val)

- Writes a 4-byte unsigned integer into the buffer in big-endian order, then advances offset by 4
- buffer : destination byte array
- offset : pointer to the current write position; incremented by 4 after write

- val : the 4-byte value to write
- byte order: [val>>24][val>>16][val>>8][val&0xFF]

int read_u8(const uint8_t *buffer, uint32_t *offset)
- Reads a single byte from the buffer at the current offset, then advances offset by 1
- buffer : source byte array
- offset : pointer to the current read position; incremented by 1 after read
- Returns the byte value as int

int read_u32(const uint8_t *buffer, uint32_t *offset)
- Reads 4 bytes from the buffer in big-endian order, then advances offset by 4
- buffer : source byte array
- offset : pointer to the current read position; incremented by 4 after read
- Returns the reconstructed 32-bit value as int

Handle Client Package

uint32_t encode_client_data(uint8_t *buffer, const PlayerAction *action)
- Serializes a PlayerAction into the buffer
- buffer : destination byte array (caller must allocate at least 6 bytes)
- action : pointer to the PlayerAction to encode
- Wire layout (6 bytes total):
- [0] playerID (1 byte, uint8_t)
- [1] move (1 byte, uint8_t, MoveType enum)
- [2..5] amount (4 bytes, uint32_t, big-endian)
- Returns the number of bytes written (always 6)

int decode_client_data(const uint8_t *buffer, uint32_t length, PlayerAction *action)
- Deserializes a PlayerAction from raw bytes
- buffer : source byte array containing the encoded action
- length : number of valid bytes in buffer; must be >= 6
- action : pointer to the PlayerAction to populate
- Returns 0 on success
- Returns -1 if length < 6 or any decoded field is negative (invalid data)

uint32_t encode_player_data(uint8_t *buffer, const Player *player)
- Serializes a single Player struct into the buffer
- buffer : destination byte array
- player : pointer to the Player to encode

int decode_player_data(const uint8_t *buffer, uint32_t length, Player *player)
- Deserializes a Player struct from raw bytes
- buffer : source byte array
- length : number of valid bytes in buffer
- player : pointer to the Player to populate

Handle Server Package

Server Data (GameState)

```
uint32_t encode_server_data(uint8_t *buffer, const GameState *gameState)
- Serializes the full GameState into a wire message
- buffer       : destination byte array (caller must allocate MAX_PAYLOAD_SIZE bytes)
- gameState    : pointer to the GameState to encode
- encode gameState and send to client
int decode_server_data(const uint8_t *buffer, uint32_t *offset, GameState *gameState)
- Deserializes a GameState from raw wire bytes
- buffer       : source byte array containing the full message
- offset       : pointer to the current read position in buffer
- gameState    : pointer to the GameState to populate
- decode receive gameState and write it into client's current gameState
```

4.3 Detailed description of the communication protocol

Protocol Constants and Settings

```
#define PORT             10160  // TCP port the server binds to and clients connect on
#define MAX_CLIENTS      6      // Maximum simultaneous connections (one per table seat)
#define BUFFER_SIZE      256    // Size of the raw read/write buffer in bytes
                                // Used for both server-side (handle_client_communication)
                                // and client-side (receive_data_from_server, input_buffer)
#define PROTOCOL_HEADER_SIZE 6  // Wire header size: 2 bytes (type) + 4 bytes (length)
                                // Note: defined as "PROTOCOL_HEADER SIZE" in com.h (missing underscore)
#define MAX_PAYLOAD_SIZE  4096  // Maximum allowed payload size in bytes per message
```

Wire Message Format

Every message exchanged between the server and client follows this 3-part layout:

```
[ type ][ length ][ payload ]
uint16_t type      // 2 bytes
uint32_t length    // 4 bytes
payload            // N bytes
Byte 0–1           Byte 2–5          Byte 6 … (6 + length - 1)
```

Field	Size	Type	Description
type	2 bytes	uint16_t	Identifies the message category (see Message Types below)
length	4 bytes	uint32_t	Number of bytes in the payload (excludes the 6-byte header)
payload	N bytes	raw bytes	The serialized message body; interpretation depends on type

All multi-byte fields are encoded in big-endian (network byte order).

Message Types

Defined in com.h as the MessageType enum:

```
MSG_TYPE_GAME_STATE    = 1   // Server → all Clients
                             // Payload: full serialized GameState
                             // Sent after every valid player action, new hand start, or stage advance
                             // Hole cards of OTHER players are masked with UNKNOW_S / UNKNOW_R

MSG_TYPE_PLAYER_ACTION = 2   // Client → Server
                             // Payload: serialized PlayerAction (6 bytes)
                             // Contains the player's chosen move and bet amount
MSG_TYPE_CHAT_MESSAGE  = 3   // Client ↔ Server (planned)
                             // Payload: UTF-8 text string
                             // Not yet implemented in game logic


MSG_TYPE_ERROR_MESSAGE = 4   // Server → Client (planned)
                             // Payload: UTF-8 error description string
                             // Intended for invalid move responses; not yet fully wired
```

Payload Encoding: PlayerAction (Client → Server)

Encoded by encode_client_data(), decoded by decode_client_data().
Total payload size: 6 bytes.

Byte 0	playerID	uint8_t	Seat index of the acting player (0–5)
Byte 1	move	uint8_t	MoveType enum value:
			0 = FOLD
			1 = CHECK
			2 = CALL
			3 = RAISE
			4 = ALL_IN
Byte 2–5	amount	uint32_t	Chip amount (big-endian)
			For FOLD / CHECK / CALL: typically 0 or the call amount
			For RAISE / ALL_IN: the total bet amount being placed

Validation on decode: decode_client_data() returns -1 if length < 6 or any field value is negative.

Payload Encoding: GameState (Server → all Clients)

Encoded by encode_server_data() (stub — not yet implemented).
When complete, the payload will serialize the full GameState struct field-by-field:
── Per player (× 6 players)

id	uint8_t	(1 byte)	Seat index
name	char[32]	(32 bytes)	Display name, null-padded
hand[0].suit	uint8_t	(1 byte)	Hole card 1 suit

```
hand[0].rank    uint8_t    (1 byte)   Hole card 1 rank
hand[0].value   uint8_t    (1 byte)   Hole card 1 numeric value
hand[1].suit    uint8_t    (1 byte)   Hole card 2 suit
hand[1].rank    uint8_t    (1 byte)   Hole card 2 rank
hand[1].value   uint8_t    (1 byte)   Hole card 2 numeric value
chips           uint32_t   (4 bytes)  Remaining chip count (big-endian)
current_bet     uint32_t   (4 bytes)  Amount bet this round (big-endian)
status          uint8_t    (1 byte)   PlayerStatus enum value
has_cards       uint8_t    (1 byte)   1 if player holds cards, 0 otherwise
── Community cards (× 5 cards)
community[i].suit    uint8_t   (1 byte)  per card
community[i].rank    uint8_t   (1 byte)
community[i].value   uint8_t   (1 byte)
── Game-level fields
playerCount     uint8_t    (1 byte)   Number of seated players
communityCount  uint8_t    (1 byte)   Number of community cards revealed so far
stage           uint8_t    (1 byte)   Stage enum: 0=PREFLOP 1=FLOP 2=TURN 3=RIVER
currentPlayer   uint8_t    (1 byte)   Seat index of the player whose turn it is
dealerIndex     uint8_t    (1 byte)   Seat index of the current dealer button
pot             uint32_t   (4 bytes)  Total chips in the pot (big-endian)
currentBet      uint32_t   (4 bytes)  Chip amount needed to call (big-endian)
minRaise        uint32_t   (4 bytes)  Minimum legal raise amount (big-endian)
handPlaying     uint8_t    (1 byte)   1 if a hand is in progress, 0 otherwise
acted[6]        uint8_t[6] (6 bytes)  1-per-seat flag: whether player has acted this round
```

Card Privacy — Hidden Information Protocol

A key protocol design is that the server never sends other players' hole cards in plaintext.
Instead, the Suit and Rank enums each provide a sentinel value for masked cards:

```
UNKNOW_S = 0   // Suit sentinel — written into card.suit when the card is hidden
UNKNOW_R = 0   // Rank sentinel — written into card.rank when the card is hidden
```

When encoding a GameState to send to player X:

- Player X's own hand cards are encoded with real suit, rank, and value.
- All other players' hand cards are encoded with suit = UNKNOW_S, rank = UNKNOW_R, value = 0.
- Community cards are always sent with real values (they are public information).

The GameState struct comment confirms this design intent:
"excludes the deck and player cards which are server-side"

---

word-size encode functions

These are the building blocks used inside all encode/decode functions:

```
void write_u8(uint8_t *buffer, uint32_t *offset, uint8_t val)
// Writes 1 byte at buffer[*offset], then increments *offset by 1
// The offset acts as a write cursor — each call advances it automatically
void write_u32(uint8_t *buffer, uint32_t *offset, uint32_t val)
```

```
// Writes 4 bytes at buffer[*offset .. *offset+3] in big-endian order
// Then increments *offset by 4
// Example — encoding 0x12345678:
//   buffer[offset+0] = 0x12   (most significant byte)
//   buffer[offset+1] = 0x34
//   buffer[offset+2] = 0x56
//   buffer[offset+3] = 0x78   (least significant byte)
int read_u8(const uint8_t *buffer, uint32_t *offset)
// Reads 1 byte from buffer[*offset], increments *offset by 1
// Returns the byte value as int
int read_u32(const uint8_t *buffer, uint32_t *offset)
// Reads 4 bytes from buffer[*offset .. *offset+3] in big-endian order
// Reassembles them into a uint32_t, increments *offset by 4
// Returns the value as int
// Reconstruction: (buf[0]<<24) | (buf[1]<<16) | (buf[2]<<8) | buf[3]
```

I/O Multiplexing — select() Usage

Both server and client use select() to monitor multiple file descriptors simultaneously without blocking threads.
Server monitors:
listen_fd — new incoming connection attempts

clients[0..5].client_fd — incoming messages from each connected player

Client monitors:
socket_fd — incoming GameState updates from the server

STDIN_FILENO (0) — keyboard input from the local user

select() blocks until at least one fd becomes readable, then the event loop inspects each fd with FD_ISSET() and dispatches to the appropriate handler. max_fd + 1 is passed as the range so the kernel only checks the file descriptors actually in use.

Error Handling in Communication

```
void error(const char *msg)
//Not implemented yet
```

---

5. Development plan and timeline

5.1 Partitioning of tasks

- Poker rules - rules.h/c, hands/h/c, specialCards.h/c
- Poker server application - server.h/c
- Poker client application - client.c
- Graphical user interface for players - playerGUI.h/c

• Graphical user interface for the server - serverGUI.h/c
• Communication protocols (data structures for data transfer via sockets) Protocol.h/Protocol.c
• Decoding/Encoding for data between the server and client Com.h/Com.c
• Handle game logic after client and server protocol game.c/game.h
• Clever Poker bot - bot.h/c
• Documentation
• System test - test.h/c
• Timer
• Special Cards - specialCards.h/c

5.2 Team member responsibilities

- Justin: Poker rules: hands.h/c, specialCards.h/c, rules.h/c
- Rishab: bot.h/c, game.h/c, documentation
- Prithvi: rules.h/c,
- Katrina: Unit testing
- Stefano: GUI
- Will: Extra Stuff, Testing
- Duc: Server.c, protocol.h/c, com.h/c

Copyright

Error Messages

Error	Meaning
ERROR: No port provided	Missing port number
ERROR, no such host	No host exists for the connecting end
ERROR connecting	Failed to connect to host
ETIMEDOUT	Connection establishment timed out without establishing a connection
ENETUNREACH	The network is not reachable from this host.

EISCONN	The socket is already connected.
ENAMETOOLONG	The length of the path argument exceeds {PATH_MAX}.
ERROR writing to socket	Fail to write to the socket
ERROR reading from socket	Fail to read from the socket

Index